

GB3D Studio 1.2 User Guide

GB3D Studio is a small scene editor and exporter for making real-time 3D Game Boy Color projects with GBDK. The editor gives you a comfortable full-resolution PC viewport while a corner preview shows the deliberately chunky renderer that the GBC actually uses.

The big viewport is for editing. The GBC preview is the truth.

1. What GB3D does

GB3D turns a scene made from OBJ models into a GBDK project containing C code, generated scene data, and the SM83 assembly triangle rasterizer.

The runtime currently supports:

- 40 x 36 logical-pixel filled 3D rendering across the full 160 x 144 screen
- multicolor flat-shaded triangles using CGB palettes
- SM83 assembly triangle rasterization
- scene objects with position and 64-step X/Y rotation
- object visibility and simple solid AABB collision
- a built-in third-person platformer player
- custom 32 x 32 player and NPC hardware sprites
- up to four scriptable NPCs
- GBScript 2 variables, conditions, loops, arithmetic, and runtime commands
- the Game Boy hardware Window layer for HUD text and bars
- an on-device performance profiler

GB3D is intentionally small. It is not Blender and it does not try to hide the limits of the Game Boy Color.

2. Requirements

For the editor:

- Python 3
- Tkinter (normally included with desktop Python)
- Pillow when importing custom 32 x 32 player or NPC artwork

Install the optional image dependency with:

```
pip install pillow
```

For exported ROMs:

- a working GBDK installation
- lcc available in your command prompt / PATH

3. Starting the editor

On Windows, double-click:

```
run.bat
```

Or run:

```
python gb3d_studio.py
```

The 1.0 editor is split into four main areas:

```
+-----+-----+-----+
| Scene / Models |      Full-resolution viewport      | Inspector |
|               |                                     | Object / Scene |
|               |                                     |   GBC   |
|               |                                     |         |
+-----+-----+-----+
| Status                |                                GBC estimate |
+-----+-----+-----+
```

The inset is a hardware-style preview of the 160 x 144 result. The large viewport uses a normal PC-side floating-point projection and global triangle sorting so editing is much easier.

4. Your first 3D scene

Step 1: Import an OBJ

Choose **Import OBJ** from the toolbar or File menu.

GB3D reads:

- vertex positions
- polygon faces (triangulated on import)
- MTL diffuse colors when available

The importer scales the model into the small integer coordinate range used by the GBC runtime.

For good performance, low-poly models are strongly recommended. A cylinder with many segments can be much more expensive than it looks on screen.

Step 2: Add an instance

Imported models appear in the **MODELS** panel. Double-click a model, or select **Add Instance**, to place it in the scene.

A model is an asset. An object is an instance of that asset with its own position, rotation, visibility, collision flag, and script.

Step 3: Select and edit it

Select an object from the hierarchy or click its geometry in the large viewport.

The Object inspector contains:

- Name
- Model
- Position X / Y / Z
- Rotation X / Y, from 0 to 63
- Visible at start
- Solid platform / collider
- Object script

Press **F** to frame the selected object.

5. Viewport controls

The default large viewport uses a detached editor camera. Moving it does not change the exported game camera.

W / S	move forward / backward
A / D	strafe left / right
Q / E	move up / down
Arrow keys	look around
Mouse wheel	move forward / backward
F	frame selected object

Enable **Game camera** in the viewport header when you want the large viewport to follow the camera/player setup that will be exported.

Other viewport toggles:

- **Grid**: editor-only world grid at Y=0
- **Wireframe**: draw model edges instead of filled editor triangles
- **GBC inset**: show/hide the hardware-style preview
- **Accurate GBC Palette**: use the engine's CGB palette approximation in the inset

6. Why there are two renderers

The Game Boy Color has no normal framebuffer suitable for this kind of 3D engine. GB3D therefore renders at 40 x 36 logical pixels. Every logical pixel corresponds to a 4 x 4 block on the LCD, and groups of four logical pixels are encoded into reusable 8 x 8 background tiles.

That is great for the GBC, but unpleasant for editing a level. Studio 1.2 therefore has two different preview paths:

Editor viewport

- full window resolution
- floating-point projection
- global painter sorting
- editor lighting for readability
- grid and selection outline
- not exported

GBC preview

- 40 x 36 logical 3D framebuffer
- runtime-style palette/shade behavior
- real 32 x 32 sprite artwork
- hardware Window UI positioning
- intended to match the exported engine closely

Always check the inset before deciding that a model or camera angle looks good on hardware.

7. Player setup

Open **Player** from the toolbar or Game menu.

The built-in platformer player can be enabled without writing code. Important settings include:

- start position
- movement speed
- jump speed
- gravity
- collider width and height
- camera distance and height
- camera pitch
- visibility
- 8 x 16 or 8 x 8 hardware-sprite mode
- custom 32 x 32 PNG artwork

Runtime controls with the player enabled:

D-Pad	<code>move</code>
Select + D-Pad	<code>orbit / look</code>
A	<code>jump</code>
B / Start	<code>available to scripts</code>

Player collision is simple AABB collision against objects marked **Solid**.

8. NPCs

Open **NPCs** to create up to four NPCs.

Each NPC has:

- a name
- 3D position
- display height
- visibility
- custom 32 x 32 sprite artwork
- its own GBScript 2 program

NPCs are hardware sprites, not triangle models. Their 3D world position is projected to a 2D sprite center every rendered frame.

Remember the real GBC sprite limits still apply. Too many sprites on one scanline can flicker or disappear just like in a normal Game Boy game.

9. Window UI

GB3D uses the Game Boy's dedicated **Window layer** for HUDs instead of drawing UI into the 3D framebuffer.

Conceptually:

Background layer -> 3D world
Window layer -> HUD / menu / dialogue box
Sprites -> player and NPCs

Open **Window UI** to configure:

- whether the Window is visible
- Window screen X and Y in physical LCD pixels
- Window background color
- text elements
- optional live numeric values
- horizontal value bars

UI element positions use 8 x 8 Window tile cells.

A bottom HUD might use:

Window X = 0
Window Y = 112

The Window is one rectangular hardware layer. It begins at its configured position and extends toward the bottom-right of the screen. It is not a collection of independent floating panels.

10. GBScript 2 basics

Scripts compile to native C during export. There is no script interpreter running on the Game Boy.

An event begins with @:

```
@start
set_sky Blue
```

```
@a
move 0 0 2
```

Persistent variables:

```
var score = 0
var timer = 60
```

Assignments and arithmetic:

```
score += 1
timer -= 1
score = score + 10
```

Conditions:

```
if score >= 10
set_sky Yellow
else
set_sky Blue
end
```

Loops are supported, but remember that they become native code that runs on an 8-bit CPU. Do not put huge loops inside `@update`.

Window UI examples:

```
@start
window_move 0, 112
window_show
ui_set_value "Score", 0
```

```
@a
ui_add_value "Score", 1
```

```
@start_button
window_toggle
```

See `SCRIPTING.md` for the complete command and expression reference.

11. Performance

GBC 3D performance depends on more than triangle count. Important costs include:

- number of vertices transformed
- number and screen size of visible triangles
- runtime object rotation

- scene object count and culling
- solid collision checks
- NPC OAM updates
- scripts that do work every tick
- VRAM upload

Choose **Performance Report** before exporting. The editor gives a rough LIGHT / MEDIUM / HEAVY estimate.

The exported ROM has a real profiler. Hold:

START + SELECT

to toggle it.

Profiler fields:

```
F  total CPU work
R  3D render / scene work
U  VRAM upload
S  scripts
P  player physics / camera
T  triangles submitted
```

Use the hardware profiler when an actual ROM is slow; it is more trustworthy than the editor estimate.

Performance tips

1. Reduce vertex count before obsessing over triangle rasterization.
2. Prefer low-sided cylinders and rounded objects.
3. Keep static objects static so export-time baking can help.
4. Split very large levels into sensible objects so coarse culling can reject them.
5. Avoid giant triangles covering most of the screen when possible.
6. Avoid expensive script loops in `@update`.
7. Test on real hardware or a representative emulator early.

12. Exporting a GBDK project

Choose **Export GBDK** and select an empty or dedicated output folder.

The export contains files such as:

```
main.c
gb3d_render.s
scene_gb3d.h
scene.gb3d
compile.bat
Makefile
README.txt
```

On Windows, if GBDK is installed and `lcc` is available, run:

`compile.bat`

The result is a `.gb` ROM targeting Game Boy Color.

If an old project was opened in a newer GB3D release, re-export it instead of compiling an export folder generated by an older Studio version.

13. Testing

Good testing targets include:

- a real Game Boy Color or compatible hardware setup
- accurate desktop Game Boy emulators
- handheld emulators that support CGB mode

A ROM that works in several unrelated emulators is useful evidence that the engine is not accidentally depending on one emulator's quirks. It still does not replace real-hardware testing for final performance decisions.

14. Common problems

lcc is not recognized

GBDK is not installed correctly or its `bin` directory is not available in `PATH`. Open a new command prompt after changing `PATH`.

The game is much slower than the editor

The large editor viewport is rendered by your PC. Look at the GBC performance estimate and the ROM profiler instead.

A model disappears near the camera

The runtime currently uses deliberately cheap near/far rejection rather than expensive general polygon clipping. Keep important geometry away from the near plane.

A model looks inside-out or faces are missing

The OBJ's face winding may be reversed. GB3D uses backface culling.

UI appears only in a rectangle at the bottom/right

That is how the Game Boy Window hardware works. Move the Window origin or hide it when it is not needed.

Sprites disappear when characters overlap vertically

You may be hitting the Game Boy's per-scanline sprite limit.

A generated project gives strange compiler errors after upgrading Studio

Re-export the project from the current editor. Generated headers and runtime templates are versioned together.

15. Keyboard shortcut reference

Ctrl+N	new project
Ctrl+O	open project
Ctrl+S	save
Ctrl+Shift+S	save as
Ctrl+D	duplicate selected object
Delete	delete selected object
F	frame selected object
WASD	move viewport camera
Q / E	move viewport camera vertically
Arrow keys	look around
Mouse wheel	move viewport camera forward/back

16. Project files

.gb3d files are human-readable JSON and contain the scene, imported model data, player/NPC sprite pixels, scripts, and Window UI setup. Imported character PNGs are embedded into the project after quantization, so the original image does not have to remain beside the project.

See GB3D_FORMAT.md for format details.

17. A sensible first project

For your first complete game, keep it intentionally tiny:

1. Import a floor and a few simple low-poly obstacles.
2. Enable the player.
3. Mark the floor and obstacles solid.
4. Add one NPC.
5. Add a two-line bottom HUD using the Window.
6. Write one or two simple button scripts.
7. Check the GBC inset.
8. Export and test the ROM.
9. Turn on the profiler if it is slow.

Once that works, expand the level gradually. On the GBC, every triangle you do not need is a tiny gift to the CPU.

Studio 1.2 performance options

Studio 1.2 replaces the broken 1.1 paired-row experiment with the stable packed-tile coverage rules plus exact quotient/remainder DDA edge walking. It also keeps native-stride CGB GDMA uploads, adds table-based fast multiplication, global triangle depth buckets, static sector rejection, Solid-only collision scans, and direct shadow-OAM position updates.

For normal play/export, leave **Profiler build (START+SELECT)** OFF. That compiles the profiler out. To measure a scene, enable it before export and hold START + SELECT in the ROM. The profiler uses the hardware Window and shows three readable rows:

```
Fxxxx Rxxxx
Uxxxx Txxxx
Sxxxx Pxxxx
```

F is total CPU work before VBlank, R is 3D scene work, U is framebuffer/UI VRAM upload, T is triangles submitted, S is scripts, and P is player physics/camera.

For a platformer with locked camera yaw/pitch, also try **Fixed Camera Fast Path**. It lets the PC exporter bake static camera rotation instead of making the GBC rotate every static vertex each frame.